Employee Data Analysis Using Pandas

Project Objective

The objective of this project is to analyze a large employee dataset using the Pandas library. We perform data loading, data cleaning, data manipulation, and data analysis to extract meaningful insights.

Dataset Description

Dataset Size

- 100,000 rows
- Multiple columns
- Contains missing values (real-world style)

Step 1. Import Required Libraries

In [56]:
```
import pandas as pd
import numpy as np
```

- In this step, we import Pandas for data analysis and NumPy for numerical operations.

Step 2: Load the Dataset

In [57]:
```
import pandas as pd
import numpy as np

df = pd.read_csv("Emp_Dataset.csv")
```

In [58]:
```
df.head()
```

Out[58]:

	Employee_ID	Name	Department	Age	Gender	Salary	Experience_Years	
0	1	Employee_1	Sales	NaN	Female	101245.0	17.0	M
1	2	Employee_2	Marketing	NaN	Male	38836.0	7.0	
2	3	Employee_3	Finance	38.0	Male	116619.0	NaN	
3	4	Employee_4	Marketing	39.0	Male	47468.0	29.0	
4	5	Employee_5	Marketing	33.0	Male	113492.0	3.0	

In [59]:
```
df.tail()
```

Out[59]:

	Employee_ID	Name	Department	Age	Gender	Salary	Experience
99995	99996	Employee_99996	Operations	25.0	Female	NaN	
99996	99997	Employee_99997	Marketing	22.0	Male	33294.0	
99997	99998	Employee_99998	Sales	49.0	Female	75407.0	
99998	99999	Employee_99999	HR	47.0	Male	124574.0	
99999	100000	Employee_100000	Sales	30.0	Female	82026.0	

- We loaded the CSV file using read_csv() and verified the data using head() and tail().

Step 3: Understand the Dataset

df.info()

df.shape()

df.columns()

df.describe()

In [60]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 100000 entries, 0 to 99999
Data columns (total 9 columns):
 #   Column            Non-Null Count   Dtype
---  ------            --------------   -----
 0   Employee_ID       100000 non-null  int64
 1   Name              100000 non-null  object
 2   Department        100000 non-null  object
 3   Age               95000 non-null   float64
 4   Gender            100000 non-null  object
 5   Salary            95000 non-null   float64
 6   Experience_Years  95000 non-null   float64
 7   City              100000 non-null  object
 8   Joining_Date      100000 non-null  object
dtypes: float64(3), int64(1), object(5)
memory usage: 6.9+ MB
```

In [61]:
```
df.shape
```

Out[61]:
```
(100000, 9)
```

In [62]:
```
df.columns
```

Out[62]:
```
Index(['Employee_ID', 'Name', 'Department', 'Age', 'Gender', 'Salary',
       'Experience_Years', 'City', 'Joining_Date'],
      dtype='object')
```

In [63]:
```
df.describe()
```

Out[63]:

	Employee_ID	Age	Salary	Experience_Years
count	100000.000000	95000.000000	95000.000000	95000.000000
mean	50000.500000	40.001558	87741.596137	17.021442
std	28867.657797	11.259238	36068.591592	10.104065
min	1.000000	21.000000	25000.000000	0.000000
25%	25000.750000	30.000000	56661.250000	8.000000
50%	50000.500000	40.000000	87704.000000	17.000000
75%	75000.250000	50.000000	118995.750000	26.000000
max	100000.000000	59.000000	149999.000000	34.000000

- This step helps us understand the structure, size, data types, and statistical summary of the dataset.

Step 4: Check Missing Values

In [64]:
```
df.isnull().sum()
```

Out[64]:
```
Employee_ID            0
Name                   0
Department             0
Age                 5000
Gender                 0
Salary              5000
Experience_Years    5000
City                   0
Joining_Date           0
dtype: int64
```

- We checked missing values to identify incomplete data which is common in real-world datasets.

Step 5: Handle Missing Values (Data Cleaning)

- Handle missing values in Age column

In [65]:
```
df["Age"].isnull().sum()

age = df["Age"].mean()

df["Age"] = df["Age"].fillna(age)
```

- Check if any missing value is remaining

In [66]:
```
df["Age"].isnull().sum()
```

Out[66]: np.int64(0)

- Handle missing values in Salary

In [67]:
```
df["Salary"] = df["Salary"].fillna(df["Salary"].median())
```

In [68]:
```
df["Salary"].isnull().sum()
```

Out[68]: np.int64(0)

- Handle missing values in Experience_Years

In [69]:
```
df["Experience_Years"] = df["Experience_Years"].fillna(df["Experience_Years"].me
```

In [70]:
```
df["Experience_Years"].isnull().sum()
```

Out[70]: np.int64(0)

- Check all missing values handle or not

In [71]:
```
df.isnull().sum()
```

Out[71]:
```
Employee_ID         0
Name                0
Department          0
Age                 0
Gender              0
Salary              0
Experience_Years    0
City                0
Joining_Date        0
dtype: int64
```

- Missing values were handled using mean, median, and default values based on the column type.

Step 6: Data Filtering

In [72]:
```
df[df["Salary"]>80000]
```

Out[72]:

	Employee_ID	Name	Department	Age	Gender	Salary	Expe
0	1	Employee_1	Sales	40.001558	Female	101245.0	
2	3	Employee_3	Finance	38.000000	Male	116619.0	
4	5	Employee_5	Marketing	33.000000	Male	113492.0	
6	7	Employee_7	Finance	36.000000	Female	131538.0	
8	9	Employee_9	Finance	36.000000	Male	142873.0	
...	...	...	...	...	...	...	
99992	99993	Employee_99993	IT	38.000000	Female	144421.0	
99993	99994	Employee_99994	Sales	56.000000	Female	87704.0	
99995	99996	Employee_99996	Operations	25.000000	Female	87704.0	
99998	99999	Employee_99999	HR	47.000000	Male	124574.0	
99999	100000	Employee_100000	Sales	30.000000	Female	82026.0	

58502 rows × 9 columns

In [73]:
```
df[(df["Department"]== "IT")  & (df["Experience_Years"]>5)]
```

Out[73]:

	Employee_ID	Name	Department	Age	Gender	Salary	Experi
5	6	Employee_6	IT	45.000000	Male	29993.0	
14	15	Employee_15	IT	43.000000	Male	35149.0	
18	19	Employee_19	IT	58.000000	Male	87704.0	
23	24	Employee_24	IT	24.000000	Female	111691.0	
31	32	Employee_32	IT	40.001558	Female	87704.0	
...	...	...	...	...	...	...	
99971	99972	Employee_99972	IT	21.000000	Female	34552.0	
99978	99979	Employee_99979	IT	33.000000	Female	140318.0	
99986	99987	Employee_99987	IT	22.000000	Male	94868.0	
99988	99989	Employee_99989	IT	27.000000	Male	134776.0	
99992	99993	Employee_99993	IT	38.000000	Female	144421.0	

14043 rows × 9 columns

- Filtering helps extract specific records based on conditions.

Step 7 Data Sorting

In [74]:
```
df.sort_values("Salary", ascending=False)
```

Out[74]:

	Employee_ID	Name	Department	Age	Gender	Salary	Experience_
50469	50470	Employee_50470	Sales	22.0	Male	149999.0	
48594	48595	Employee_48595	IT	31.0	Male	149999.0	
70786	70787	Employee_70787	HR	47.0	Male	149998.0	
67209	67210	Employee_67210	IT	22.0	Female	149994.0	
74650	74651	Employee_74651	HR	21.0	Male	149992.0	
...	...	...	...	...	...	...	
1948	1949	Employee_1949	Sales	49.0	Male	25005.0	
85347	85348	Employee_85348	Operations	29.0	Female	25004.0	
94202	94203	Employee_94203	Operations	48.0	Female	25003.0	
24035	24036	Employee_24036	Finance	24.0	Female	25000.0	
65698	65699	Employee_65699	Marketing	33.0	Female	25000.0	

100000 rows × 9 columns

- Sorting was performed to rank employees based on salary.

Step 8: GroupBy Analysis

Average salary per department

In [75]:
```
df.groupby("Department")["Salary"].mean()
```

Out[75]:
```
Department
Finance       87348.805064
HR            88096.916526
IT            87876.311030
Marketing     87923.824688
Operations    87802.346179
Sales         87384.991237
Name: Salary, dtype: float64
```

In [76]:
```
df.groupby("City")["Employee_ID"].count()
```

Out[76]:
```
City
Bangalore    16709
Chennai      16535
Delhi        16599
Hyderabad    16716
Mumbai       16600
Pune         16841
Name: Employee_ID, dtype: int64
```

- GroupBy is used to perform department-wise and city-wise analysis.

Step 9: Data Transformation

Add New Column

In [77]:
```
df["Salary_yearly"] = df["Salary"] * 12
```

In [78]:
```
df.head()
```

Out[78]:

	Employee_ID	Name	Department	Age	Gender	Salary	Experience_Year
0	1	Employee_1	Sales	40.001558	Female	101245.0	17.
1	2	Employee_2	Marketing	40.001558	Male	38836.0	7.
2	3	Employee_3	Finance	38.000000	Male	116619.0	17.
3	4	Employee_4	Marketing	39.000000	Male	47468.0	29.
4	5	Employee_5	Marketing	33.000000	Male	113492.0	3.

- New columns were created to derive additional insights.

Step 10: Export Cleaned Data

In [79]:
```
df.to_csv("Cleaned_Emp_Data.csv", index=False)
```

- The cleaned and processed dataset was exported for future use.

Conclusion

In this project, I successfully analyzed a large dataset using Pandas. We performed data cleaning, filtering, grouping, and statistical analysis to extract meaningful insights.